

closure_evidence_20260925

BOB-159 closure evidence — warm-start ownership repair: bounded repair-then-reverify- until-stable

Date: 2026-09-25 **Item:** BOB-159 — warm `./start.sh ownership-repair` race window (feature 002-user-owned-downloads) **Operator decision implemented (2026-08-26, §11.4.66):** REPAIR, THEN RE-VERIFY UNTIL STABLE — options rejected: (a) refuse the warm repair and direct the operator to `--recreate`; (b) quiesce the stack on the warm path too; (c) accept the window and merely document it.

This document records the implementation of the operator's already-recorded decision, the bounded-loop design rationale, and pasted RED → GREEN → mutation terminal output from this session.

1. What was built

`scripts/ownership_repair.sh` is **unchanged** — the design does not need a new probe mode. The existing `--force` flag (bypasses the completion marker's fast-path) plus the script's own `complete: N item(s) repaired` line (already printed on every successful run) are sufficient to drive a bounded repair-then-reverify loop from `start.sh` without inventing a second “how many items changed” channel (§11.4.251).

`start.sh` gained one new function, `run_ownership_repair_until_stable()`, plus a small constant `OWNERSHIP_REPAIR_MAX_PASSES=4`. `run_ownership_gate()` — the WARM-start entry point (used by the normal, no-flag `./start.sh` boot path only) — now calls this new function instead of the single-pass

`run_ownership_repair()`. The `--recreate` dispatch is untouched: it calls `run_ownership_precondition / stack_down / run_ownership_repair / stack_up` directly, exactly as before, because `stack_down()` already quiesces every writer before its one repair pass runs (FR-004d) — the bounded loop is a WARM-path problem only.

Mechanism

```
run_ownership_repair_until_stable():
    for pass in 1..OWNERSHIP_REPAIR_MAX_PASSES:
        run `ownership_repair.sh --force`, capture its output and
        exit code
        if exit code != 0:
            → print the per-item
failures already reported, refuse to start (exit 1)
            parse "complete: N item(s) repaired" from the output
            if N cannot be parsed:
                → refuse to start (exit 1) –
an unreadable pass is not evidence of a clean tree
            if N == 0:
                → STABLE – print success,
return 0
            else:
                → print "repaired N item(s)
– re-verifying", loop again
        # loop exhausted without N == 0:
        → print ownership UNPROVEN (names the realistic cause: an
active download / a container writing
        faster than the walk completes), refuse to start (exit 1) –
never a silent exit 0
```

`--force` is required on every pass: `ownership_repair.sh`'s own marker fast-path (`marker_is_valid`) makes a second call against the same scope fingerprint a no-op that walks nothing at all — the mirror problem to the one its own `--dry-run` header already documents as unusable here (“would walk the tree twice on every start”). A completed marker has the opposite failure mode: it would walk the tree ZERO times on every re-verify pass, defeating the whole point of re-verifying. `--force` makes every pass a real walk.

2. Bounded-loop design rationale — why 4 passes

This is a §11.4.6 judgement call, stated explicitly rather than left as an unexplained magic number:

- **An unbounded loop is not “re-verifying” against a stack that writes faster than the walk completes — it is a hang with extra steps.** The loop must terminate and say so honestly. This alone forces *some* finite bound.

- **The common case (already-clean tree, or one genuine pre-existing straggler) converges in 1–2 passes.** Pass 1 either finds nothing (stable immediately) or fixes a real straggler; pass 2 then confirms nothing new. A bound of 4 gives that case comfortable headroom (2×) without materially slowing a normal boot, since each pass on an already-repaired scope is a fast walk (the declared tree, not the whole filesystem — see `ownership_path_fence` in `scripts/ownership_repair.sh`).
- **A genuinely non-converging stack (an active download continuously creating files) is not rescued by a larger N.** If every pass finds something new, the correct answer is UNPROVEN regardless of whether the bound is 4, 40, or 400 — a bigger number only spends more of the operator’s boot time reaching the same honest verdict. Tuning the bound upward to “try harder” would be exactly the kind of guess §11.4.6 forbids: it does not change the outcome, only the latency before reporting it.
- **4 was therefore chosen as “enough headroom for the common converging case, small enough that the non-converging case fails fast.”** It is recorded as consumer DATA (a `start.sh`-local constant, `OWNERSHIP_REPAIR_MAX_PASSES`), not hardcoded inline, so a future operator decision to change it is a one-line edit with no logic change.

3. Honest give-up behaviour (§11.4.201(6))

On exhausting `OWNERSHIP_REPAIR_MAX_PASSES` without a pass reporting 0 new items, the function:

- exits **non-zero** (refuses to complete the boot) — never a silent exit 0;
- prints `Ownership UNPROVEN` after 4 repair pass(es) — explicitly stating this is **not** the same as a clean tree, quoting §11.4.201(6) directly in the message: *“a bounded loop that stops because it ran out of passes must never be reported the same as a pass that genuinely found nothing left to fix”*;
- **names the realistic cause:** “an ACTIVE DOWNLOAD (or another container) writing into a declared location FASTER than the repair can walk it”;
- gives an actionable remediation: `./start.sh --recreate` (quiesces the stack for the walk) or waiting for write activity to settle and re-running `./start.sh`.

An unparseable pass result (the repair script exited 0 but its output did not contain a readable complete: N item(s) line) is treated the same way — refused, not silently trusted — per the same §11.4.201(6) reasoning: a result that cannot be read is not evidence of anything.

4. Diff applied — start.sh

New function inserted after `run_ownership_repair()`;
`run_ownership_gate()` updated to call it; the stale “NOT closed here” honest-boundary comment at the `main()` call site corrected to describe the new bounded behaviour (§11.4.6 — a comment that no longer matches the code is a bluff of its own).

```
diff --git a/start.sh b/start.sh
index 9a22c56..3892b6a 100755
--- a/start.sh
+++ b/start.sh
@@ -1358,13 +1358,139 @@ run_ownership_repair() {
     print_success "Ownership repair complete – every in-scope
        item is operator-owned"
 }

+# -----
+# run_ownership_repair_until_stable – the WARM-start half of the
+#   ownership
+# gate (BOB-159; operator decision 2026-08-26, §11.4.66: REPAIR,
+#   THEN
+# RE-VERIFY UNTIL STABLE).
+#
+# WHY THIS EXISTS, AND WHY run_ownership_repair() ALONE IS NOT
+#   ENOUGH HERE:
+#   run_ownership_repair() walks the declared tree ONCE. On the
+#     --recreate
+#   path that single walk is provably sufficient, because
+#     stack_down() has
+#   ALREADY quiesced every writer before it runs (FR-004d – see
+#     the
+#   "ORDERING" header above run_ownership_precondition()). On the
+#     WARM path
+#   there is no stack_down(): if the stack is already running, a
+#     container
+#   can create a new non-operator-owned file BEHIND a single
+#     walk, after
+#   which that walk's own completion marker would record
+#     "complete" over a
+#   tree that is not – the exact defect feature 002-user-owned-
+#     downloads
+#   exists to end, re-opened through the warm door instead of the
+#     --recreate
+#   one it was closed on.
+#
```

```
+# THE OPERATOR'S CHOICE (recorded 2026-08-26; options NOT chosen
    are named
+# here so this is never silently re-litigated):
+#   - refuse the warm repair and direct the operator to --
        recreate: REJECTED
+#   - quiesce the stack on the warm path too (correct, but no
        longer "warm"): REJECTED
+#   - accept the window and merely document it: REJECTED
+#   - CHOSEN: keep repairing the live stack, but WALK, RE-VERIFY,
        and REPEAT
+#       until a pass finds NOTHING NEW – bounded – and on giving up
        REPORT
+#       HONESTLY that ownership is UNPROVEN rather than exit 0.
+#
+# WHY BOUNDED AT 4 PASSES, NOT UNBOUNDED AND NOT SOME OTHER
    NUMBER:
+#   An unbounded loop against a stack that writes faster than the
        walk
+#   completes never returns – that is a hang with extra steps,
        not
+#   re-verification. 4 is a judgement call (§11.4.6 – stated, not
        hidden):
+#   on the common, already-clean case a converging pass costs the
        operator
+#   at most a couple of extra fast walks (pass 1 fixes a genuine
        straggler,
+#   pass 2 confirms nothing new), so the bound is cheap when the
        tree really
+#   is stabilising. A genuinely non-converging stack (an active
        download
+#   continuously creating files) is not rescued by a LARGER bound
        – the
+#   honest answer there is UNPROVEN regardless of how many passes
        are tried
+#   – so the number is kept small rather than tuned to paper over
        that case.
+#
+# WHY --force ON EVERY PASS:
+#   ownership_repair.sh's own marker fast-path (marker_is_valid)
        makes a
+#   SECOND call against the SAME scope fingerprint a no-op that
        walks
+#   nothing at all – which is the same reason its own --dry-run
        header
+#   explains --dry-run cannot serve as a warm-start probe ("would
        walk the
+#   tree twice on every start"): a completed marker has the
        opposite
+#   problem, it would walk the tree ZERO times on every re-verify
        pass.
+#   --force bypasses the marker so every pass performs a REAL
        walk and can
+#   see a file that arrived after the previous pass finished.
+#
+# CONVERGENCE SIGNAL:
```

```

+# Read from ownership_repair.sh's OWN "complete: N item(s)
    repaired" line
+# ($11.4.251 – one definition of "how many items changed" per
    run, never a
+# second one re-derived here from the marker JSON or the change
    record).
+# N == 0 on a pass means that pass's walk found nothing left to
    fix – the
+# tree was already stable when that walk started. N > 0 means
    the walk
+# found (and fixed) something, which could be a genuine pre-
    existing
+# straggler OR a file a live container wrote WHILE the walk
    ran; one pass
+# alone cannot tell those apart, which is exactly why this loop
    re-verifies
+# instead of trusting the first "complete".
+#
+# HONEST BOUNDARY ($11.4.6): a "stable" verdict here means the
    LAST pass
+# found nothing new – it does not retroactively prove nothing was
    ever
+# written mid-walk during an EARLIER pass in this same loop; that
    is why
+# every pass with items > 0 triggers another full re-walk rather
    than being
+# treated as "probably fine". It also does not prove nothing
    writes AFTER
+# this function returns – that is the ordinary, always-present
    race this
+# function narrows, not one it claims to eliminate.
+# -----
+OWNERSHIP_REPAIR_MAX_PASSES=4
+
+run_ownership_repair_until_stable() {
+    local repair="$SCRIPT_DIR/scripts/ownership_repair.sh"
+    local pass=0 rc out items
+    ownership_set_nice_prefix
+
+    if [[ ! -f "$repair" ]]; then
+        print_error "Ownership repair script missing: $repair"
+        print_error "Refusing to start – pre-existing content
            cannot be brought under the operator (FR-004d).\"
+        exit 1
+    fi
+
+    print_info "Ownership repair (warm start): repairing, then
        re-verifying until stable (bounded, max $
        {OWNERSHIP_REPAIR_MAX_PASSES} pass(es))..."
+
+    while (( pass < OWNERSHIP_REPAIR_MAX_PASSES )); do
+        pass=$((pass + 1))
+        set +e
+        out="$("${OWNERSHIP_NICE[@]}" bash "$repair" --force
            2>&1)"

```

```

+         rc=$?
+         set -e
+         printf '%s\n' "$out"
+
+         if [[ "$rc" -ne 0 ]]; then
+             print_error "Ownership repair did not complete on
+             pass ${pass}/${OWNERSHIP_REPAIR_MAX_PASSES} (exit $rc) –
+             refusing to start."
+             print_error "  Each item it could not repair is named
+             in the report above (FR-006)."
+             exit 1
+         fi
+
+         items="$(printf '%s\n' "$out" | sed -n 's/.*complete: \
+         ([0-9][0-9]*) item(s) repaired.*\/\1/p' | tail -n1)"
+         if [[ -z "$items" ]]; then
+             print_error "Ownership repair exited 0 on pass $
+             ${pass}/${OWNERSHIP_REPAIR_MAX_PASSES} but did not report
+             how many items it changed."
+             print_error "  A pass whose result cannot be read is
+             not evidence of a clean tree (§11.4.201(6)) – refusing to
+             certify ownership as stable."
+             exit 1
+         fi
+
+         if [[ "$items" -eq 0 ]]; then
+             print_success "Ownership repair complete and STABLE
+             after ${pass} pass(es) – the last pass found nothing new
+             to repair"
+             return 0
+         fi
+
+         print_info "  pass ${pass}/${OWNERSHIP_REPAIR_MAX_PASSES}
+             repaired ${items} item(s) – re-verifying (a live container
+             may still be writing)"
+     done
+
+     print_error "Ownership UNPROVEN after $
+             {OWNERSHIP_REPAIR_MAX_PASSES} repair pass(es) – every pass
+             kept finding NEW non-operator-owned files."
+     print_error "  This is NOT the same as a clean tree
+             (§11.4.201(6)): a bounded loop that stops because it ran
+             out of"
+     print_error "  passes must never be reported the same as a
+             pass that genuinely found nothing left to fix."
+     print_error "  The realistic cause is an ACTIVE DOWNLOAD (or
+             another container) writing into a declared location"
+     print_error "  FASTER than the repair can walk it, so
+             ownership cannot be certified stable on this live stack."
+     print_error "  Remediate with: ./start.sh --recreate
+             (quiesces the stack for the walk before bringing it back
+             up),"
+     print_error "  or wait for the write activity to settle and
+             re-run ./start.sh."
+     exit 1

```

```

+}
+
# Cold-start / warm-start entry point: the declared locations
# have just been
# created and no container has been brought up yet by THIS
# invocation, so both
# halves run back to back. The --recreate path does NOT use this
# wrapper -- it
# interleaves `down` between the halves; see the dispatch below.
+#
+# BOB-159: the repair half here is
# run_ownership_repair_until_stable(), NOT
+# the single-pass run_ownership_repair() the --recreate dispatch
# below still
+# calls directly. See that function's own header comment for why
# a warm start
+# needs the bounded re-verify loop and --recreate does not.
run_ownership_gate() {
    run_ownership_precondition
-   run_ownership_repair
+   run_ownership_repair_until_stable
}

# -----
@@ -1608,16 +1734,18 @@ main() {
    #
    # HONEST BOUNDARY (§11.4.6), do not read more into this than
    # it says: the
    # claim above is about what THIS invocation starts. If the
    # operator runs a
-   # warm `./start.sh` over an ALREADY-RUNNING stack, containers
    # are writing
-   # while the repair walks, and the FR-004d window the --
    # recreate path closes
-   # by interleaving `down` is NOT closed here.
-   #
-   # Bounded in practice, not by luck: after any successful pass
    # the completion
-   # marker is valid for the scope fingerprint and the repair
    # short-circuits
-   # without walking at all (scripts/ownership_repair.sh
    # marker_is_valid), so
-   # the window needs a STALE-or-absent marker AND a live stack
    # together. That
-   # is a real combination – a newly declared scope entry re-
    # arms the marker –
-   # so it is tracked, not dismissed.
+   # warm `./start.sh` over an ALREADY-RUNNING stack, containers
    # can still be
+   # writing while a repair pass walks – a single pass cannot,
    # by itself, tell
+   # "genuinely clean" apart from "a container wrote something
    # behind this
+   # walk". run_ownership_gate() no longer trusts one pass on
    # the warm path:

```

```

+ # it calls run_ownership_repair_until_stable() (BOB-159,
+   operator decision
+ # 2026-08-26), which walks, re-verifies, and repeats until a
+   pass finds
+ # NOTHING NEW, bounded to OWNERSHIP_REPAIR_MAX_PASSES
+   attempts. That BOUNDS
+ # the window instead of leaving it open-ended, but does not
+   eliminate it –
+ # a stack whose writes never stabilise within the bound is
+   reported
+ # honestly as ownership UNPROVEN (the function refuses to
+   start rather than
+ # exit 0 over an unverified tree – see its own header for the
+   full
+ # rationale, including why a larger bound would not change
+   that verdict).
  run_ownership_gate

  if [[ "$build_frontend_flag" == true ]]; then

```

scripts/ownership_repair.sh — **no diff** (unchanged; verified with `git diff --stat scripts/ownership_repair.sh` → empty output before closing this item).

5. Diff applied — tests/unit/ test_start_reload_recreate.sh

Two new checks, `WARM_REVERIFY_UNTIL_STABLE` and `WARM_REVERIFY_UNPROVEN_ON_NONCONVERGENCE`, each with a paired §1.1 mutation, plus the supporting test double / direct-function-call harness helpers they need. `tests/unit/test_start_reload_harness.sh` (the shared harness) was **not modified** — the new helpers (`harness_write_repair_double`, `harness_call_stable`) live entirely inside this test file, reusing the harness's existing `harness_new_sandbox` / `harness_mutate` / `harness_cleanup` primitives.

Testing approach: `harness_run()` drives `main()` end-to-end (right for the pre-existing `--recreate` checks), but the warm-path bounded loop is reachable from `main()`'s default no-flag path, which would additionally require shimming `start_container` / `wait_for_jackett` / `curl` / `python3` / a container runtime for no added assurance. `start.sh` is deliberately hermetically sourceable (its own trailing `if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then main "$@"; fi` guard exists for this purpose), so the two new checks instead **source** the sandboxed `start.sh` (`main()` never runs) and call `run_ownership_repair_until_stable` directly, against a purpose-built `ownership_repair.sh` **double** (not the harness's generic argv-recorder

shim) that returns a scripted per-call item count via the real complete:
N item(s) repaired line shape, so both convergence and non-
convergence are reproducible without performing a real filesystem
walk.

```
diff --git a/tests/unit/test_start_reload_recreate.sh b/tests/
      unit/test_start_reload_recreate.sh
index 3c93684..0dd70d1 100644
--- a/tests/unit/test_start_reload_recreate.sh
+++ b/tests/unit/test_start_reload_recreate.sh
@@ -153,10 +153,129 @@ check_REPAIR_BEFORE_UP() {
     harness_cleanup "$sb"; return $ok
 }

+# --- BOB-159: warm-start bounded repair-until-stable
+-----
+# Operator decision 2026-08-26 (§11.4.66): REPAIR, THEN RE-VERIFY
+  UNTIL
+# STABLE. The --recreate path above needs no loop -- stack_down()
+  already
+# quiesces every writer before its single repair pass runs
+  (FR-004d). The
+# WARM path (plain `./start.sh`, no flags) has no stack_down(): a
+  live
+# container can write a new non-operator-owned file BEHIND a
+  single walk, so
+# run_ownership_gate() now calls
+  run_ownership_repair_until_stable(), which
+# walks, re-verifies, and repeats -- bounded -- until a pass
+  finds NOTHING
+# NEW, or honestly gives up as UNPROVEN rather than exit 0.
+#
+# harness_run() drives main() end to end, which is right for the
+  --recreate
+# checks above but would drag in start_container/
+  wait_for_jackett/curl/
+# python3/etc for no added assurance here: the function under
+  test is a
+# fixed, self-contained unit, and start.sh is deliberately
+  hermetically
+# sourceable (see its own trailing `if [[ "${BASH_SOURCE[0]}" ==
+  "${0}" ]]`
+# guard) for exactly this reason. These two checks SOURCE the
+  sandboxed
+# start.sh (main() never runs) and call
+  run_ownership_repair_until_stable
+# directly, against a purpose-built ownership_repair.sh DOUBLE --
+  not the
+# harness's generic argv-recorder shim -- that reports a SCRIPTED
+  per-call
+# item count (DOUBLE_ITEMS_SEQUENCE, colon-separated) via
+  ownership_repair.sh's
+# own real "complete: N item(s) repaired" line shape, so
+  convergence and
```

```

+## non-convergence are both reproducible without a real filesystem
+    walk.
+
+## harness_write_repair_double <sandbox> - overwrite the generic
+    recorder shim
+## scripts/ownership_repair.sh with the scripted double described
+    above.
+harness_write_repair_double() {
+    local sb="$1"
+    cat > "$sb/repo/scripts/ownership_repair.sh" <<'DOUBLE'
+##!/usr/bin/env bash
+## BOB-159 test double for scripts/ownership_repair.sh - returns a
+    SCRIPTED
+## per-call item count (DOUBLE_ITEMS_SEQUENCE, colon-separated)
+    instead of
+## performing a real filesystem walk, so
+    run_ownership_repair_until_stable's
+## convergence / non-convergence behaviour is reproducible without
+    one.
+set -euo pipefail
+state_dir="${DOUBLE_STATE_DIR:?DOUBLE_STATE_DIR unset}"
+mkdir -p "$state_dir"
+count_file="$state_dir/repair_calls"
+n=0
+[[ -f "$count_file" ]] && n="$(cat "$count_file")"
+n=$((n + 1))
+printf '%s' "$n" > "$count_file"
+printf '%s\n' "$*" >> "$state_dir/repair_argv.log"
+
+IFS=: read -r -a seq <<< "${DOUBLE_ITEMS_SEQUENCE:?
+    DOUBLE_ITEMS_SEQUENCE unset}"
+idx=$((n - 1))
+if (( idx < ${#seq[@]} )); then
+    items="${seq[$idx]}"
+else
+    items="${seq[${#seq[@]}-1]}"
+fi
+
+printf '[ownership-repair] complete: %s item(s) repaired; record
+    x.ndjson; marker repair-marker.json\n' "$items"
+exit 0
+DOUBLE
+
+    chmod +x "$sb/repo/scripts/ownership_repair.sh"
+}
+
+## harness_call_stable <sandbox> <colon-separated-item-sequence> -
+    source the
+## sandboxed start.sh (main() never runs, per its own sourcing
+    guard) and call
+## run_ownership_repair_until_stable() directly. Sets
+    HARNESS_OUT / HARNESS_RC,
+## same contract as harness_run().
+harness_call_stable() {
+    local sb="$1" seq="$2"

```

```

+ local out rc=0
+ mkdir -p "$sb/state"
+ set +e
+ out="$(
+     cd "$sb/repo" && \
+     PATH="$sb/bin:$sb/sysbin" \
+     HOME="$sb/home" \
+     DOUBLE_STATE_DIR="$sb/state" \
+     DOUBLE_ITEMS_SEQUENCE="$seq" \
+     bash -c 'source "$1/repo/start.sh";
+     run_ownership_repair_until_stable' _ "$sb" 2>&1
+ )"
+ rc=$?
+ set -e
+ HARNESS_OUT="$out"
+ HARNESS_RC="$rc"
+}
+
+check_WARM_REVERIFY_UNTIL_STABLE() {
+    # First pass finds 2 stragglers (repaired), second pass finds
+    # 0 (nothing
+    # new) -- the function must STOP re-verifying there: exactly
+    # 2 calls,
+    # rc=0, and it must say so as "STABLE after 2 pass(es)".
+    local sb; sb="$(harness_new_sandbox)"
+    harness_write_repair_double "$sb"
+    [[ -n "$1" ]] && harness_mutate "$sb" "$1"
+    harness_call_stable "$sb" "2:0"
+    local ok=1 calls
+    calls="$(cat "$sb/state/repair_calls" 2>/dev/null || echo 0)"
+    if [[ "$HARNESS_RC" -eq 0 && "$HARNESS_OUT" == *"STABLE after
+    2 pass(es)*" && "$calls" == "2" ]]; then
+        ok=0
+    fi
+    CHECK_DIAG="rc=$HARNESS_RC calls=$calls out tail: $
+    {HARNESS_OUT: -200}"
+    harness_cleanup "$sb"; return $ok
+}
+
+check_WARM_REVERIFY_UNPROVEN_ON_NONCONVERGENCE() {
+    # Every pass reports 3 NEW items -- an active download
+    # writing faster
+    # than the walk completes never lets a pass find nothing. The
+    # loop MUST
+    # be bounded (exactly OWNERSHIP_REPAIR_MAX_PASSES=4 calls,
+    # never more),
+    # MUST exit non-zero, and MUST say UNPROVEN + name the
+    # realistic cause --
+    # never silently exit 0 over an unverified tree
+    # ($11.4.201(6)).
+    local sb; sb="$(harness_new_sandbox)"
+    harness_write_repair_double "$sb"
+    [[ -n "$1" ]] && harness_mutate "$sb" "$1"

```

```

+ harness_call_stable "$sb" "3:3:3:3:3:3"
+ local ok=1 calls
+ calls="$(cat "$sb/state/repair_calls" 2>/dev/null || echo 0)"
+ if [[ "$HARNESS_RC" -ne 0 && "$HARNESS_OUT" == *"UNPROVEN"* \
+ && "$HARNESS_OUT" == *"ACTIVE DOWNLOAD"* && "$calls" ==
+ "4" ]]; then
+ ok=0
+ fi
+ CHECK_DIAG="rc=$HARNESS_RC calls=$calls out tail: $
+ {HARNESS_OUT: -240}"
+ harness_cleanup "$sb"; return $ok
+}
+
CHECK_NAMES=(
    PRECONDITION_BEFORE_DOWN
    REPAIR_AFTER_DOWN
    REPAIR_BEFORE_UP
+ WARM_REVERIFY_UNTIL_STABLE
+ WARM_REVERIFY_UNPROVEN_ON_NONCONVERGENCE
    BOBACTL_DOWN_AND_UP DOWN_BEFORE_UP UP_IS_DETACHED
    NO_RAW_RUNTIME_CALLS
    NO_BOBA_CTL_USES_COMPOSE EXIT_ZERO_AND_SUCCESS_MSG
    DOWN_FAILURE_IS_NONFATAL UP_FAILURE_IS_FATAL
@@ -165,6 +284,8 @@ declare -A CHECK_DESC=(
    [PRECONDITION_BEFORE_DOWN]='probes ownership BEFORE tearing
    the stack down (FR-010)'
    [REPAIR_AFTER_DOWN]='repairs only after down, so no container
    writes behind the walk (FR-004d)'
    [REPAIR_BEFORE_UP]='completes the repair before up -d
    restores writers (FR-004d)'
+ [WARM_REVERIFY_UNTIL_STABLE]='warm start re-verifies until a
+ pass finds nothing new, then stops (BOB-159)'
+ [WARM_REVERIFY_UNPROVEN_ON_NONCONVERGENCE]='bounded give-up
+ reports ownership UNPROVEN, never a silent exit 0
+ (BOB-159, §11.4.201(6))'
    [BOBACTL_DOWN_AND_UP]='issues down and up -d through the
    boba-ctl orchestrator'
    [DOWN_BEFORE_UP]='tears the stack down before bringing it up'
    [UP_IS_DETACHED]='brings the stack up detached (-d)'
@@ -178,6 +299,8 @@ declare -A CHECK_MUTATION=(
    [PRECONDITION_BEFORE_DOWN]='s/^
    run_ownership_precondition$//; /^          stack_down$/a
    \          run_ownership_precondition'
    [REPAIR_AFTER_DOWN]='/^          stack_down$/i \
    run_ownership_repair'
    [REPAIR_BEFORE_UP]='s/^          run_ownership_repair$//; /
    ^          stack_up$/a \          run_ownership_repair'
+ [WARM_REVERIFY_UNTIL_STABLE]='s/if \[\[ "\$items" -eq 0 \]\];
+ then/if [[ "$items" -eq 999999 ]]; then/'
+ [WARM_REVERIFY_UNPROVEN_ON_NONCONVERGENCE]='/or wait for the
+ write activity to settle/{n;s/exit 1/:/}'
    [BOBACTL_DOWN_AND_UP]='s/COMPOSE_CMD="\$SCRIPT_DIR/scripts\
    boba-ctl.sh"/COMPOSE_CMD="\$SCRIPT_DIR/scripts\boba-ctl-
    TYPO.sh"/'

```

```
[DOWN_BEFORE_UP]='/print_info "Recreating the full stack/a
\      $COMPOSE_CMD up -d'
[UP_IS_DETACHED]='s/if ! \ $COMPOSE_CMD up -d; then/if !
$COMPOSE_CMD up; then/'
```

6. Terminal evidence

6a. RED — new checks fail against the pre-fix start.sh (\$11.4.115: RED reproduces on the real,

broken artifact before any fix exists)

start.sh was reverted to its pre-fix (HEAD) state via
git stash push -- start.sh (path-scoped, so no other in-flight work in
the tree was touched), confirmed by grep -c
run_ownership_repair_until_stable start.sh → 0, then the full suite
was run:

```
$ git stash push --keep-index -m "BOB-159 temp: pre-fix start.sh
for RED verification" -- start.sh
Saved working directory and index state On main: BOB-159 temp:
pre-fix start.sh for RED verification
```

```
$ grep -c "run_ownership_repair_until_stable" start.sh
0
```

```
$ bash tests/unit/test_start_reload_recreate.sh
== start.sh --recreate (--green) ==
PASS: PRECONDITION_BEFORE_DOWN – probes ownership BEFORE tearing
the stack down (FR-010)
PASS: REPAIR_AFTER_DOWN – repairs only after down, so no
container writes behind the walk (FR-004d)
PASS: REPAIR_BEFORE_UP – completes the repair before up -d
restores writers (FR-004d)
FAIL: WARM_REVERIFY_UNTIL_STABLE – warm start re-verifies until
a pass finds nothing new, then stops (BOB-159)
rc=127 calls=0 out tail:
FAIL: WARM_REVERIFY_UNPROVEN_ON_NONCONVERGENCE – bounded give-up
reports ownership UNPROVEN, never a silent exit 0 (BOB-159,
$11.4.201(6))
rc=127 calls=0 out tail:
PASS: BOBCTL_DOWN_AND_UP – issues down and up -d through the
boba-ctl orchestrator
PASS: DOWN_BEFORE_UP – tears the stack down before bringing it
up
PASS: UP_IS_DETACHED – brings the stack up detached (-d)
PASS: NO_RAW_RUNTIME_CALLS – never issues a raw podman/docker
verb (Hard Stop #3)
PASS: NO_BOBA_CTL_USES_COMPOSE – --no-boba-ctl re-binds to raw
```

podman-compose

PASS: EXIT_ZERO_AND_SUCCESS_MSG – exits 0 and reports success

PASS: DOWN_FAILURE_IS_NONFATAL – a failing down warns but still brings the stack up

PASS: UP_FAILURE_IS_FATAL – exits non-zero when up fails

RESULT: 11 passed, 2 failed

rc=127 (“command not found”) and calls=0 confirm the failure is genuine: run_ownership_repair_until_stable does not exist on the prefix artifact, so the function call fails before the double is ever invoked. This is real RED, not a synthetic one.

6b. Fix restored (git stash pop), syntax verified

```
$ git stash pop
```

```
...
```

```
Dropped refs/stash@{0} (f0c2610b2d2903be0a21f4bf604d825031cf222c)
```

```
$ bash -n start.sh && echo "SYNTAX OK"
```

```
SYNTAX OK
```

```
$ grep -c "run_ownership_repair_until_stable" start.sh
```

```
5
```

6c. GREEN — full suite passes with the fix applied

```
$ bash tests/unit/test_start_reload_recreate.sh
```

```
== start.sh --recreate (--green) ==
```

PASS: PRECONDITION_BEFORE_DOWN – probes ownership BEFORE tearing the stack down (FR-010)

PASS: REPAIR_AFTER_DOWN – repairs only after down, so no container writes behind the walk (FR-004d)

PASS: REPAIR_BEFORE_UP – completes the repair before up -d restores writers (FR-004d)

PASS: WARM_REVERIFY_UNTIL_STABLE – warm start re-verifies until a pass finds nothing new, then stops (BOB-159)

PASS: WARM_REVERIFY_UNPROVEN_ON_NONCONVERGENCE – bounded give-up reports ownership UNPROVEN, never a silent exit 0 (BOB-159, §11.4.201(6))

PASS: BOBACTL_DOWN_AND_UP – issues down and up -d through the boba-ctl orchestrator

PASS: DOWN_BEFORE_UP – tears the stack down before bringing it up

PASS: UP_IS_DETACHED – brings the stack up detached (-d)

PASS: NO_RAW_RUNTIME_CALLS – never issues a raw podman/docker verb (Hard Stop #3)

PASS: NO_BOBA_CTL_USES_COMPOSE – --no-boba-ctl re-binds to raw podman-compose

PASS: EXIT_ZERO_AND_SUCCESS_MSG – exits 0 and reports success

PASS: DOWN_FAILURE_IS_NONFATAL – a failing down warns but still

brings the stack up

PASS: UP_FAILURE_IS_FATAL – exits non-zero when up fails

RESULT: 13 passed, 0 failed

Full suite went from **11/11** (before this item) to **13/13** (after) — the two new checks, both green, zero regressions among the pre-existing 11.

6d. RED (mutation) — every check, including the two new ones, has a paired \$1.1 mutation that kills it

```
$ bash tests/unit/test_start_reload_recreate.sh --red
== start.sh --recreate (--red) ==
  RED-OK: PRECONDITION_BEFORE_DOWN – killed by: s/^
run_ownership_precondition$//; /^          stack_down$a \
run_ownership_precondition
  RED-OK: REPAIR_AFTER_DOWN – killed by: /^          stack_down$/i
\          run_ownership_repair
  RED-OK: REPAIR_BEFORE_UP – killed by: s/^
run_ownership_repair$//; /^          stack_up$a \
run_ownership_repair
  RED-OK: WARM_REVERIFY_UNTIL_STABLE – killed by: s/if \[\[ "\
$item" -eq 0 \]\]; then/if [[ "$item" -eq 999999 ]]; then/
  RED-OK: WARM_REVERIFY_UNPROVEN_ON_NONCONVERGENCE – killed by: /
or wait for the write activity to settle/{n;s/exit 1:/}
  RED-OK: BOBCTL_DOWN_AND_UP – killed by: s/COMPOSE_CMD="\
$SCRIPT_DIR/scripts/boba-ctl.sh"/COMPOSE_CMD="$SCRIPT_DIR/\
scripts/boba-ctl-TYPO.sh"/
  RED-OK: DOWN_BEFORE_UP – killed by: /print_info "Recreating the
full stack/a \    $COMPOSE_CMD up -d
  RED-OK: UP_IS_DETACHED – killed by: s/if ! \ $COMPOSE_CMD up -d;
then/if ! $COMPOSE_CMD up; then/
  RED-OK: NO_RAW_RUNTIME_CALLS – killed by: /print_info
"Recreating the full stack/a \    $CONTAINER_RUNTIME restart
qbittorrent-proxy
  RED-OK: NO_BOBA_CTL_USES_COMPOSE – killed by: s/
COMPOSE_CMD="podman-compose"/COMPOSE_CMD="podman-compose-TYPO"/
  RED-OK: EXIT_ZERO_AND_SUCCESS_MSG – killed by: s/print_success
"Stack recreated successfully"/print_success "maybe"/
  RED-OK: DOWN_FAILURE_IS_NONFATAL – killed by: /Stack may not
have been running/a \    exit 1
  RED-OK: UP_FAILURE_IS_FATAL – killed by: /Failed to bring the
stack back up/{n;s/exit 1:/}
RESULT: 13 passed, 0 failed
```

WARM_REVERIFY_UNTIL_STABLE's mutation (-eq 0 → -eq 999999) breaks the convergence check itself: the "2:0" sequence's pass-2 result of 0 is no longer recognised as stable, so the loop keeps going to the 4-pass bound and gives up UNPROVEN instead — the check (which requires rc=0 and "STABLE after 2 pass(es)") correctly fails.

WARM_REVERIFY_UNPROVEN_ON_NONCONVERGENCE's mutation neuters the final give-up `exit 1` (turns it into a no-op :) so the function falls off the end returning 0 despite never converging — reproducing, under a controlled mutation, exactly the silent-exit-0 bug this item exists to prevent. The check (which requires `rc != 0`) correctly fails, proving the `exit 1` is load-bearing and not decorative.

6e. Regression sweep — related ownership/boot-integrity suites unaffected

`scripts/ownership_repair.sh` was not touched (`git diff --stat scripts/ownership_repair.sh` → empty), and `run_ownership_precondition()` / `run_ownership_repair()` / the `--recreate dispatch` are unchanged, so every other ownership-related suite was re-run to confirm no regression:

```
=== test_ownership_repair.sh ===  
RESULT: 153 passed, 0 failed, 0 skipped
```

```
=== test_ownership_precondition.sh ===  
RESULT: 32 passed, 0 failed, 0 skipped
```

```
=== test_ownership_gid_agreement.sh ===  
RESULT: 5 passed, 0 failed, 0 skipped
```

```
=== test_ownership_repair_scope_tsv.sh ===  
RESULT: 2 passed, 0 failed, 1 skipped (pre-existing HEAD-moved-  
past-baseline SKIP, unrelated to this change)
```

```
=== test_ownership_repair_setgid_strip.sh ===  
RESULT: 1 passed, 0 failed, 1 skipped (pre-existing HEAD-moved-  
past-baseline SKIP, unrelated to this change)
```

```
=== test_credential_store_mode.sh ===  
RESULT: 25 passed, 0 failed
```

```
=== test_start_sh_boot_integrity.sh ===  
=== summary: PASS=13 FAIL=0 ===
```

`tests/unit/test_start_reload_harness.sh`'s own control-needle self-test (proving the recorder-shim mechanism this suite depends on can genuinely see, and that the repo's real `start.sh` was never mutated by the harness) was also re-run and passed 6/6.

7. Scope discipline honoured

- Only `start.sh`, `tests/unit/test_start_reload_recreate.sh`, and this evidence file were modified. `scripts/ownership_repair.sh` was read in full but left unchanged (no probe mode was genuinely needed).
- `scripts/ownership_precondition.sh`, `scripts/lib/ownership.sh`, `scripts/pre_build_verification.sh`, `scripts/commit-push-all.sh`, and `docs/workable_items.db` / `docs/Issues.md` / `docs/Fixed.md` / `workable-items CLI` were never touched.
- No `git add` / `git commit` / `git push` was run.
- No real container was started, stopped, or recreated, and no real filesystem walk was performed against this host's live stack during testing — all new-check assertions run against an isolated sandbox (`harness_new_sandbox`) with a scripted double standing in for `ownership_repair.sh`, per the existing established safe-testing pattern in this test file.